Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ВЕБ-АУДИОПЛЕЕР «Meloidia»

Студент гр. 350504 Смольский А.В.

Проверил: Внук О.М.

Минск 2025

ВВЕДЕНИЕ

Meloidia — веб-приложение для комфортного прослушивания музыки и управления аудиоколлекцией прямо в браузере. Продукт предоставляет простой и интуитивный плеер с возможностью воспроизведения треков с сервера и сохранения их для оффлайн-прослушивания, а также базовый набор инструментов для создания и редактирования персональных плейлистов и загрузки аудиофайлов на сервер. Интерфейс ориентирован на удобство и минимализм, чтобы пользователь мог быстро находить и организовывать музыку без лишних настроек.

В то же время Meloidia не предназначена для организации live-стримов, не реализует сложные механизмы персонализированных рекомендаций и не служит платформой для коммерческой продажи или лицензирования контента. В текущей версии отсутствует система аутентификации и детализированного разграничения прав, а интеграция с внешними сервисами и мобильные нативные приложения не входят в рамки проекта. Эти ограничения задают чёткие границы функциональности и помогают сосредоточиться на надёжном воспроизведении, оффлайн-режиме и удобстве управления плейлистами.

Приложение ориентировано на индивидуальных пользователей и небольшие сообщества, которым важна простота и стабильность воспроизведения; дальнейшее развитие может включать улучшение работы с локальным кэшем, оптимизацию производительности и расширение инструментов управления библиотекой, при этом любые новые возможности будут внедряться с учётом сохранения лёгкости использования и защиты пользовательских данных.

1. ТРЕБОВАНИЯ ПОЛЬОВАТЕЛЯ

1.1. Программные интерфейсы

Продукт будет взаимодействовать со следующими внешними и внутренними системами:

− postgreSQL: Реляционная база данных для хранения информации о пользователях, аудиотреках, плейлистах и истории прослушивания;

− Spring Boot REST API: Серверный интерфейс, предоставляющий доступ к функциональности приложения через HTTP-запросы (загрузка, потоковая передача, поиск и скачивание музыки);

− файловая система сервера: Хранилище аудиофайлов, доступное через backend-приложение с проверкой прав доступа;

− Electron: Десктопная оболочка, обеспечивающая запуск клиентского интерфейса вне браузера с доступом к локальному хранилищу и файловой системе;

− JWT (JSON Web Token): Механизм аутентификации и авторизации пользователей при доступе к защищённым ресурсам;

Основной стек внутренних сервисов:

Для серверной части: Java, Spring Boot, Spring Security, Spring Data JPA, postgreSQL.

Для клиентской части: JavaScript, Electron, Axios.

1.2. Интерфейс пользователя

Взаимодействие с пользователем будет происходить через интерфейс веб-приложения, реализованного с использованием Electron. Интерфейс обеспечивает доступ к основным функциям продукта и организован в виде набора страниц, каждая из которых отвечает за определённый аспект пользовательского опыта. Мокапы приложения представлены на рисунках 1.1-1.4

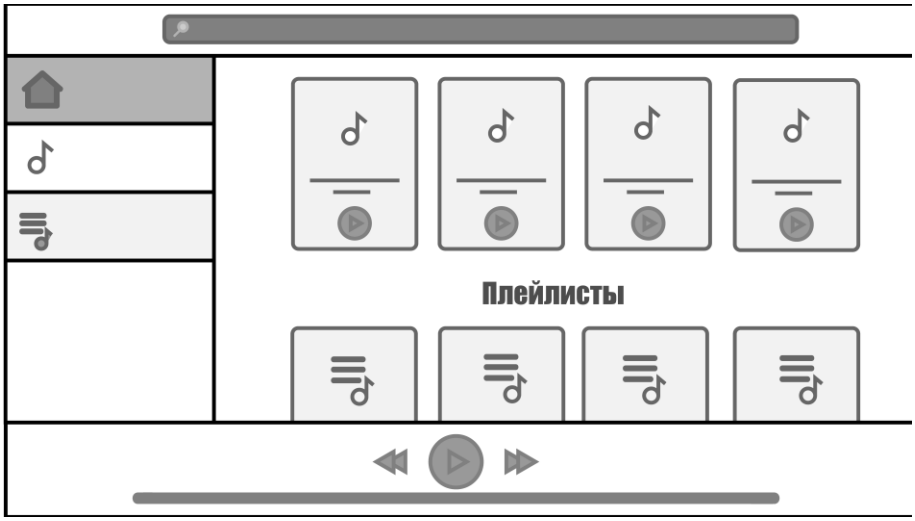


Рисунок 1.1 – Страница «Главная»

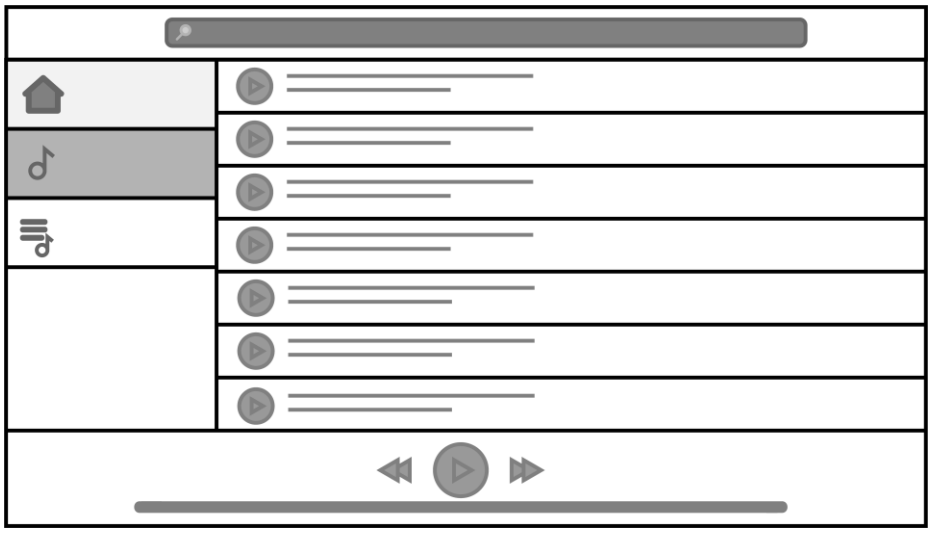


Рисунок 1.2 – Страница «Моя музыка»

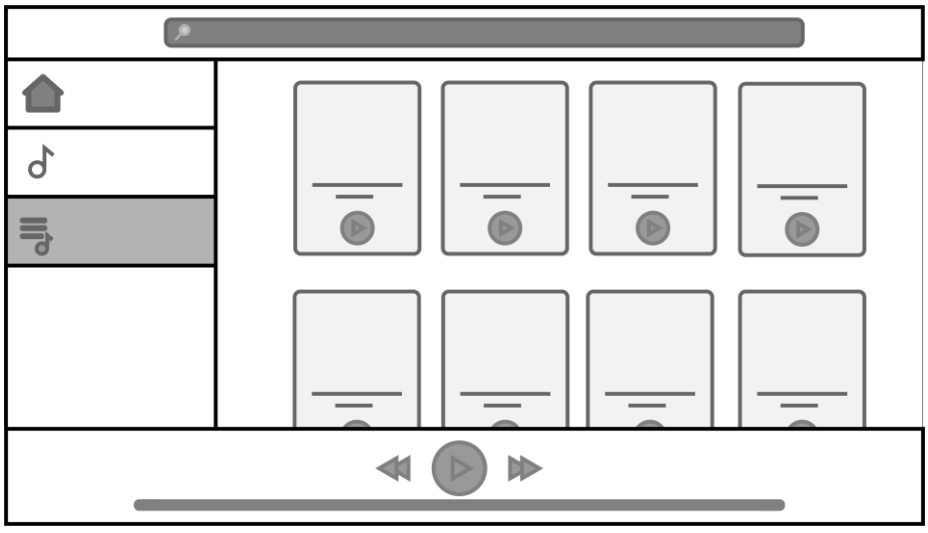


Рисунок 1.3 – Страница «Плейлисты»

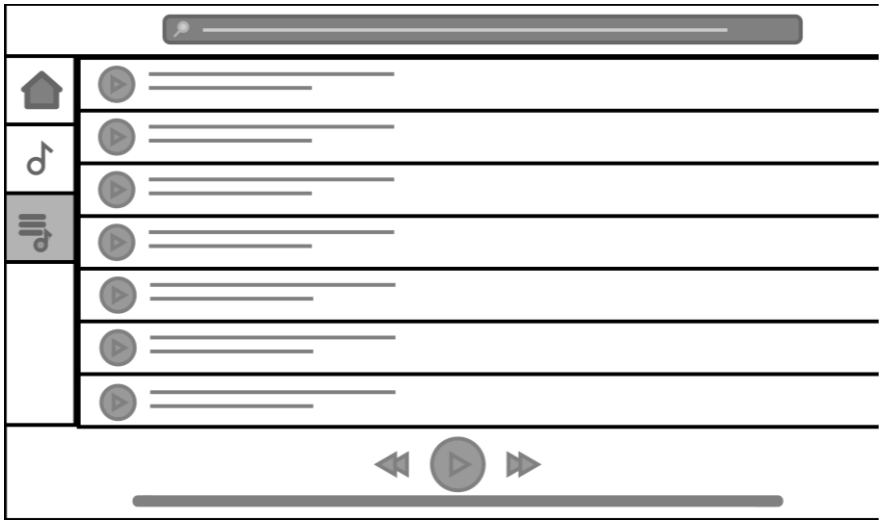


Рисунок 1.4 – Страница «Поиск»

Также будут представлены вспомогательные элементы интерфейса в виде: аудиоплеера, который доступен на всех страницах, поддерживает воспроизведение, перемотку, паузу, повтор и плейлисты и навигационного меню обеспечивающего переход между разделами.

1.3. Характеристики пользователей

Обычные пользователи – конечные слушатели, использующие приложение для загрузки, прослушивания и управления треками. Обладают базовой технической грамотностью, уверенно работают с ПК. Интерфейс адаптирован под интуитивное взаимодействие без необходимости специальной подготовки.

Зарегистрированные авторы – пользователи, публикующие оригинальные треки. Имеют средний уровень технической подготовки, знакомы с аудиоформатами и авторскими правами. Используют систему для загрузки контента, управления доступом и отслеживания статистики.

1.4. Предположения и зависимости

На требования, изложенные в настоящем документе, могут оказывать влияние следующие факторы:

− увеличение количества пользователей и объёма обрабатываемых данных;

− ужесточение требований к безопасности и защите персональных данных;

− обновление версий применяемых технологий и компонентов;

− необходимость интеграции с внешними сервисами и платформами;

2. СИСТМНЫЕ ТРЕБОВАНИЯ

2.1. Функциональные требования

Программное средство должно обеспечивать выполнение следующих функций:

- регистрация и аутентификация пользователей с использованием логина и пароля;
- загрузка аудиофайлов пользователями с сохранением метаданных (название, исполнитель, жанр);
- хранение аудиофайлов в файловой системе сервера с привязкой к пользователю;
- поиск треков по ключевым параметрам: названию, исполнителю;
- потоковая передача аудиофайлов с поддержкой перемотки и паузы;
- формирование и редактирование пользовательских плейлистов;
- отображение списка загруженных треков и плейлистов на соответствующих страницах интерфейса;
- кэширование аудиофайлов на клиентской стороне для оффлайн-доступа;
- защита доступа к функциональности с использованием JWT-токенов;
- обработка ошибок и отображение системных уведомлений в интерфейсе;

2.2. Нефункциональные требования

2.2.1. Атрибуты качества

Надёжность — система должна обеспечивать устойчивую работу при длительной эксплуатации, корректно обрабатывать ошибки и исключения. Надёжность измеряется количеством отказов за единицу времени, а также временем восстановления после сбоя.

Безопасность — система должна обеспечивать защиту пользовательских данных, предотвращать несанкционированный доступ и обеспечивать безопасную передачу информации. Безопасность оценивается наличием механизмов аутентификации, шифрования, контроля доступа и соответствием нормативным требованиям (например, GDPR).

Производительность — система должна обеспечивать приемлемое время отклика при выполнении основных операций (поиск, загрузка, воспроизведение). Производительность измеряется временем ответа API, скоростью обработки запросов и устойчивостью к нагрузке.

Удобство использования — интерфейс должен быть интуитивно понятным, доступным для пользователей с базовой технической подготовкой. Удобство измеряется через пользовательские опросы, количество ошибок взаимодействия и время освоения интерфейса.

Сопровождаемость — система должна быть легко модифицируема и расширяема. Сопровождаемость оценивается по полноте документации, читаемости кода и времени, необходимому для внесения изменений.

3. ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРЕКТИРОВАНИЕ СИСТЕМЫ

3.1 Модель предметной области

Модель предметной области описывает ключевые сущности веб-аудиоплеера — Пользователей, Исполнителей, Песни, Плейлисты и Чат — и связи между ними. Она показывает, как данные хранятся, взаимосвязываются и используются при выполнении бизнес-логики приложения. Центральным объектом является сущность Song (Песня), которая относится к исполнителю (Artist) и может находиться в нескольких плейлистах. Плейлист (Playlist) создаётся пользователем (User) и может быть публичным или приватным; приватный плейлист при необходимости защищается параметрами доступа.

Пользователи также взаимодействуют друг с другом через встроенный чат (Chat): чат может включать несколько участников и множество сообщений (Message), что реализует связь «многие-ко-многим» между пользователями и чатами и связь «один-ко-многим» между чатом и сообщениями. Для обеспечения обмена данными и уведомлений в реальном времени в архитектуре предусмотрена логическая модель ConnectionModel (неперсистентная): она управляет активными WebSocket-соединениями пользователей, хранит сведения о подключениях и обеспечивает мгновенную доставку сообщений, синхронизацию статусов воспроизведения и оповещений между клиентами.

Глоссарий представлен в таблице 3.1.

Диаграмма классов, иллюстрирующая перечисленные сущности и связи, представлена на рисунке 3.1.

Таблица 3.1 – Глоссарий

Термин	Определение
Пользователь (User)	Аккаунт системы; владеет плейлистами
Исполнитель (Artist)	Музыкант/группа; связан с песнями
Песня (Song)	Музыкальная запись; может быть сохранена офлайн
Плейлист (Playlist)	Коллекция песен пользователя
Элемент плейлиста (PlaylistItem)	Связь песня–плейлист с позицией
Кеш песен (Cache<Song>)	LRU-кеш для ускорения доступа и офлайн-хранения
Сервис песен (SongService)	CRUD, поиск, импорт каталога, офлайн-загрузка
Сервис плейлистов (PlaylistService)	CRUD по плейлистам
Сервис исполнителей (ArtistService)	CRUD по исполнителям

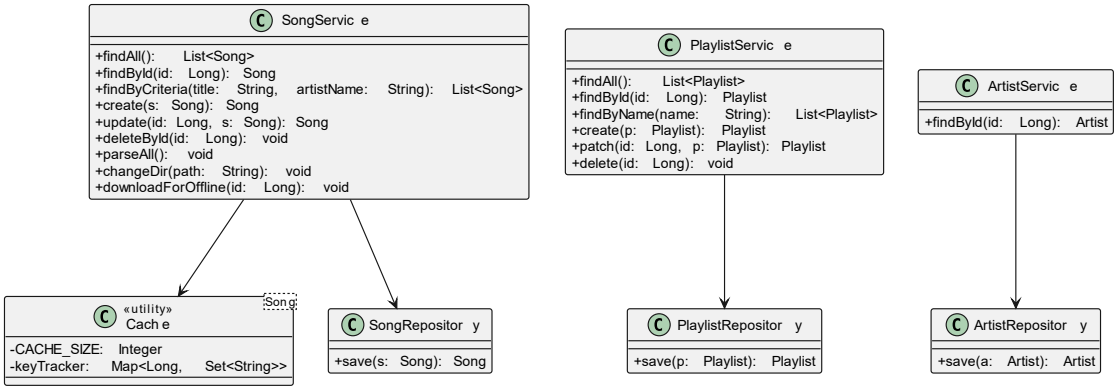


Рисунок 3.1 – Диаграмма классов

3.2 Use-Case представление системы

Use Case представление отражает взаимодействие пользователей с веб-аудиоплеером и показывает, какие функции доступны каждому актёру. Оно строится на основе системных требований и характеристик пользователей, и служит связующим звеном между требованиями и архитектурой.

Актёры системы:

Пользователь — основной актёр, который авторизуется в системе, управляет плейлистами, воспроизводит музыку, сохраняет треки в кеш для офлайн-прослушивания.

Гость — неавторизованный посетитель, имеющий доступ только к публичным плейлистам и песням, без возможности сохранять их офлайн или редактировать.

Система кеша — внутренний актёр, обеспечивающий сохранение и извлечение треков из локального хранилища устройства.

Сервер приложения — обеспечивает бизнес-логику, обработку запросов и взаимодействие с базой данных PostgreSQL.

Основные варианты использования:

1. Воспроизведение трека Пользователь выбирает песню в интерфейсе. Система проверяет наличие офлайн-копии в кеше. Если она есть — воспроизведение идёт локально. Если нет — трек загружается с сервера. Пользователь может управлять воспроизведением (play, pause, stop, seek).

2. Сохранение трека офлайн Пользователь нажимает кнопку «Сохранить офлайн». Система загружает файл с сервера, сохраняет его в локальном кеше и отмечает песню как доступную для офлайн-прослушивания. При недостатке места или ошибке сети система сообщает об ошибке.

3. Создание и управление плейлистами Пользователь создаёт новый плейлист, задаёт название и описание, добавляет песни, меняет их порядок

или удаляет. Плейлист сохраняется в базе данных и доступен для дальнейшего использования.

4. Поиск песен и исполнителей Пользователь вводит запрос (название песни или имя исполнителя). Система выполняет поиск по базе данных и возвращает список результатов. При пустом результате отображается сообщение «Ничего не найдено».

5. Импорт каталога песен (системная операция) Администратор запускает функцию импорта. Сервер сканирует указанную директорию, извлекает метаданные файлов, обновляет базу данных и кеш. Это обеспечивает актуальность медиатеки.

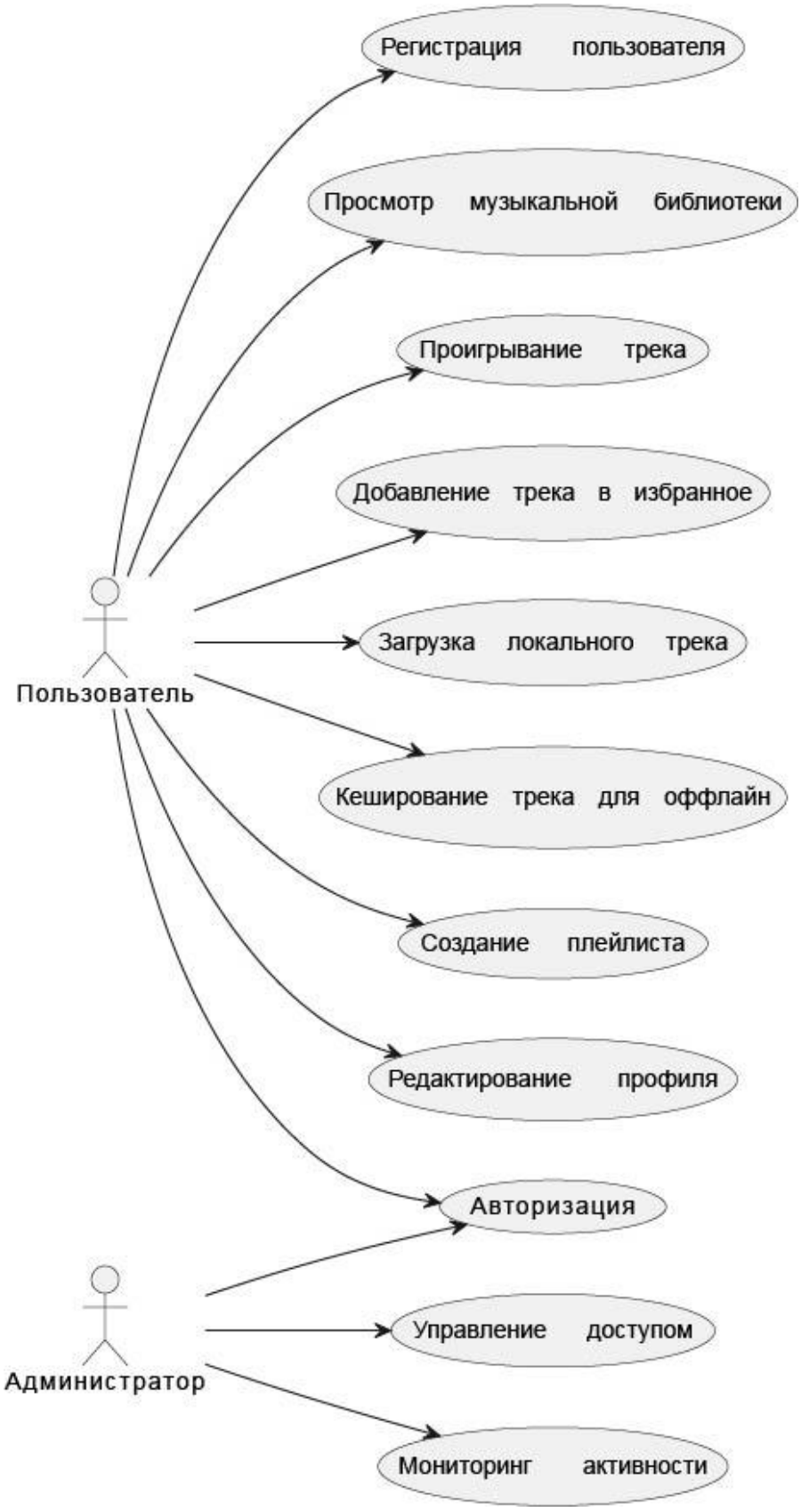


Рисунок 3.2 – Use-Case диаграмма

3.3 Ключевой пользовательский сценарий «Воспроизведение трека»

Ниже представлен сценарий «Воспроизведение трека»:

1. Пользователь открывает список песен или плейлист в интерфейсе.
2. Пользователь кликает по выбранному треку.
3. Система проверяет, есть ли офлайн-копия трека в локальном кеше устройства.
4. Если копия найдена, система готовит локальный файл к воспроизведению.
5. Если копии нет, клиент отправляет запрос на сервер для получения потока данных.
6. Сервер проверяет права доступа пользователя и наличие трека в базе данных.
7. Сервер возвращает поток аудиоданных клиенту.
8. Аудиоплеер (на клиенте) инициализирует воспроизведение: запускает декодер, начинает проигрывание.
9. Интерфейс отображает состояние PLAYING, показывает прогресс-бар, обложку и название трека.
10. Пользователь может управлять воспроизведением:

− Нажать «Пауза» – состояние меняется на PAUSED.

− Нажать «Стоп» – состояние меняется на STOPPED.

− Перемотка (seek) – аудиоплеер изменяет позицию воспроизведения.

Альтернативные потоки:

1. А1 – Ошибка сети: если соединение с сервером отсутствует, система предлагает воспроизвести офлайн-копию (если она есть).
2. А2 – Нет офлайн-копии и нет сети: система выводит сообщение «Трек недоступен».
3. А3 – Ошибка файла: если локальный файл повреждён, система удаляет его из кеша и предлагает загрузить заново.

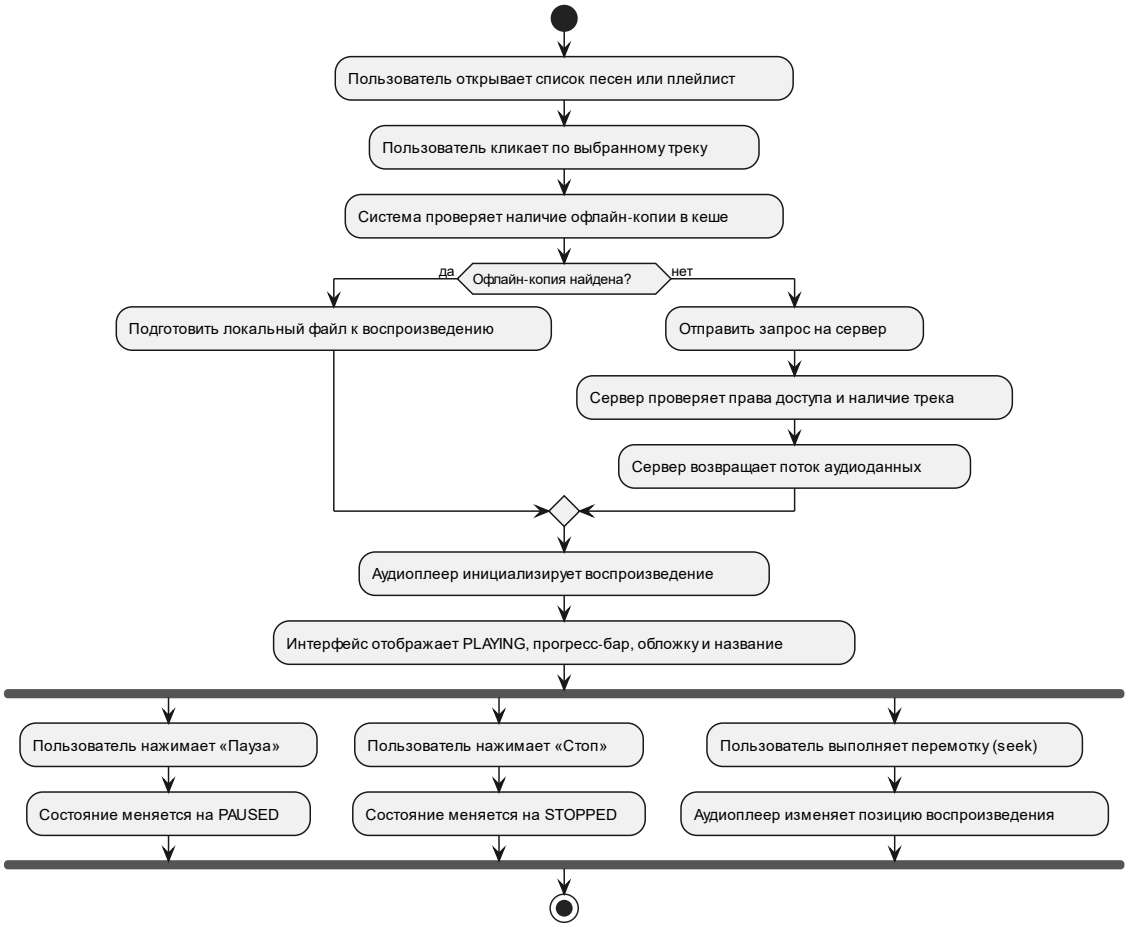


Рисунок 3.3 – диаграмма активности

3.4 Диаграмма последовательности

Диаграмма последовательности используется для того, чтобы наглядно показать взаимодействие между актёрами и объектами системы во времени. Она фиксирует порядок сообщений и вызовов, которые происходят при выполнении конкретного сценария, например воспроизведения трека или сохранения его офлайн.

Диаграмма последовательности отражает взаимодействие актёров: Пользователь → UI → Кеш → Сервер → Аудиоплеер, и показывает, как система обеспечивает воспроизведение трека как из локального кеша, так и через поток с сервера.

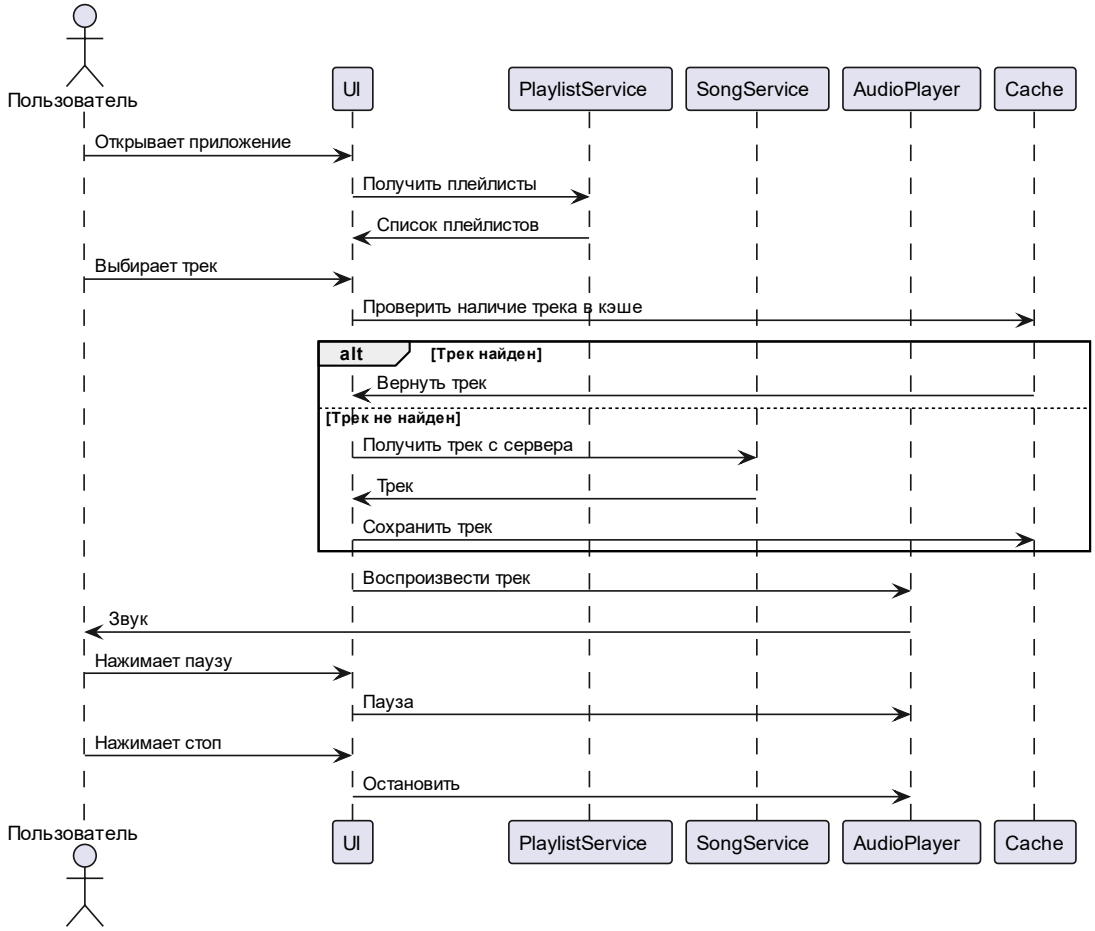


Рисунок 3.4 – диаграмма последовательности

4. ТЕСТИРОВАНИЕ УЧЕБНОГО ПРОЕКТА

4.1. Введение (Introduction)

Цель тест-плана: Обеспечить качество настольного аудиоплеера, реализованного на Electron и React, с серверной частью на Spring REST и базой данных PostgreSQL, через систематическое тестирование функциональных и нефункциональных требований.

Область тестирования: Полное тестирование фронтенда (Electron + React) и бэкенда (Spring REST), взаимодействия с файловой системой пользователя, потоковой и оффлайн-проигрывания музыки, функционала плейлистов и добавления треков пользователями.

4.2. Объект тестирования (Test Items)

Краткое описание проекта: Десктопное приложение-аудиоплеер, которое позволяет проигрывать музыку с сервера и сохранять треки локально для оффлайн-прослушивания. Пользователи могут создавать и редактировать плейлисты, загружать и добавлять музыку. Аутентификация отсутствует — доступ к функционалу не требует логина.

Основные компоненты:

− фронтенд: Electron + React.js;

− бэкенд: Spring Framework REST API;

− база данных: PostgreSQL;

− хранение локально: файловая система пользователя (сохранение треков, кэш);

Атрибуты качества:

− функциональная корректность — воспроизведение, сохранение, управление плейлистами;

− надежность — устойчивость к сбоям сети и ошибкам файловой системы;

− удобство использования — понятный интерфейс плеера и управления плейлистами;

− безопасность — защита локальных файлов и предотвращение инъекций через загружаемые метаданные;

− производительность — быстрая загрузка, низкая задержка при воспроизведении, экономное использование памяти;

4.3. Риски (Risk Issues)

Технические риски:

− проблемы с синхронизацией потокового воспроизведения и локального кэша при переключении онлайн/offline;

− ошибки работы с файловой системой: права доступа, повреждённые файлы, недостаток места;

− неполная валидация загружаемых треков (формат, метаданные) — возможные сбои плеера;

− утечки памяти и утилиты Electron при длительном воспроизведении;

Бизнес-риски:

− потеря пользовательских данных (плейлисты, локально сохранённые треки);

− плохой UX при нестабильной сети — пользователи не смогут слушать музыку;

− нарушение авторских прав при неконтролируемой загрузке контента пользователями;

Ограничения:

− отсутствие механизма аутентификации усложняет аудит и разграничение прав;

− ограниченное время на тестирование и отсутствие точной копии продакшен-сервера;

4.4. Аспекты тестирования (Features to be Tested)

Функциональный набор:

− воспроизведение музыки: потоковое воспроизведение с сервера; пауза, перемотка, переключение треков;

− оффлайн режим: сохранение треков на диск; воспроизведение сохранённых файлов при отсутствии сети;

− плейлисты: создание, переименование, удаление, добавление/удаление треков; порядок воспроизведения;

− загрузка музыки пользователями: загрузка файлов на сервер; валидация форматов и метаданных;

− интеграция фронтенд бэкенд: корректные REST-запросы, обработка ошибок сети;

− работа с файловой системой: проверка путей, прав доступа, обработка ошибок записи/чтения.

4.5. Подходы к тестированию (Test Approach)

Функциональное тестирование:

− воспроизведение музыки: потоковое воспроизведение с сервера; пауза, перемотка, переключение треков;

− оффлайн режим: сохранение треков на диск; воспроизведение сохранённых файлов при отсутствии сети;

− плейлисты: создание, переименование, удаление, добавление/удаление треков; порядок воспроизведения;

Нефункциональное тестирование:

− тестирование производительности: замеры времени старта, latency воспроизведения, профилирование памяти (Chrome DevTools, Node profiler);

− тестирование устойчивости: симуляция обрывов сети, тесты на восстановление после сбоев;

− тестирование безопасности: проверка валидации загружаемых файлов, защита от path traversal при сохранении;

− тестирование удобства использования: ручное тестирование UX, сценарии с реальными пользователями.

4.6. Инструменты тестирования:

Бэкенд:

− Junit

− Mockito

− Spring Boot Test

− coverage для измерения покрытия кода

API тестирование:

Postman для ручного тестирования

4.7. Представление результатов (Pass / Fail Criteria)

Результаты тестирования приложения выглядят слдующим образом

1. 90% покрытие кода unit-тестами для критического функционала.
2. Все высокоприоритетные тест-кейсы пройдены.
3. Отсутствие критических багов.
4. Соответствие SRS требованиям.

4.8. Тестовые сценарии (Test Cases)

Группа 1: Аутентификация и авторизация

TC-PLAY-001: Успешное потоковое воспроизведение

Назначение: Проверить потоковое воспроизведение трека с сервера. Предусловия: Приложение запущено; сеть доступна; в списке треков есть хотя бы один трек на сервере.

Шаги:

1.Открыть приложение.

2.Выбрать трек на сервере.

3.Нажать «Play».

Ожидаемый результат: Звук воспроизводится; индикатор прогресса двигается; ошибок нет.

TC-PLAY-002: Пауза и возобновление

Назначение: Проверить паузу и возобновление воспроизведения. Предусловия: Трек воспроизводится.

Шаги:

1.Нажать «Pause».

2.Подождать 3–5 секунд.

3.Нажать «Play» (Resume).

Ожидаемый результат: Пауза останавливает звук; Resume продолжает воспроизведение с текущей позиции.

TC-OFFLINE-001: Сохранение трека локально

Назначение: Проверить сохранение трека локально и воспроизведение в оффлайн.

Предусловия: Доступен трек на сервере; на диске достаточно места. Шаги:

1.Выбрать трек;

2.Нажать «Сохранить для оффлайн»;

3.Дождаться завершения сохранения;

4.Отключить сеть;

5.Воспроизвести сохранённый трек.

Ожидаемый результат: Трек сохраняется локально; при отсутствии сети воспроизводится из локального хранилища без ошибок.

TC-OFFLINE-002: Повреждённый файл

Назначение: Проверить поведение при попытке воспроизвести повреждённый локальный файл.

Предусловия: В локальную папку поместить повреждённый аудиофайл.

Шаги:

1.Выбрать повреждённый файл в приложении;

2.Нажать «Play».

Ожидаемый результат: Плеер корректно обрабатывает ошибку: не падает, отображает понятное сообщение об ошибке и предлагает удалить/переустановить файл.

Группа 2: Плейлисты и управление контентом

TC-PL-001: Создание плейлиста

Назначение: Проверить создание плейлиста. Предусловия: Приложение запущено.

Шаги:

1.Открыть раздел «Плейлисты»;

2.Нажать «Создать плейлист»;

3.Ввести имя и сохранить.

Ожидаемый результат: Плейлист создаётся и отображается в списке плейлистов.

TC-PL-002: Добавление трека в плейлист

Назначение: Проверить добавление трека в плейлист и сохранение порядка.

Предусловия: Существует плейлист; в библиотеке есть треки.

Шаги:

1. Открыть плейлист;

2. Нажать «Добавить трек»;

3. Выбрать трек(и);

4. Сохранить.

Ожидаемый результат: Трек(и) появляются в плейлисте; порядок соответствует выбранному.

TC-PL-003: Удаление плейлиста

Назначение: Проверить удаление плейлиста без удаления локальных треков.

Предусловия: Существует плейлист с локально сохранёнными треками.

Шаги:

1. Открыть список плейлистов;

2. Выбрать плейлист;

3. Нажать «Удалить плейлист»;

4. Подтвердить удаление.

Ожидаемый результат: Плейлист удалён; локально сохранённые файлы остаются на диске, если не выбрано явное удаление файлов.

TC-UPLOAD-001: Загрузка трека пользователем

Назначение: Проверить загрузку трека пользователем (поддерживаемые форматы).

Предусловия: Сервер доступен; пользовательская функция загрузки включена.

Шаги:

1. В интерфейсе загрузки выбрать файл формата mp3 или wav;

2. Нажать «Загрузить»;

3. Дождаться ответа сервера.

Ожидаемый результат: Сервер принимает файл; метаданные валидируются; трек появляется в списке и доступен для воспроизведения.

TC-UPLOAD-002: Неподдерживаемый формат

Назначение: Проверить обработку неподдерживаемого формата при загрузке.

Предусловия: Сервер доступен.

Шаги:

1. Попытаться загрузить файл неподдерживаемого формата (например .exe или .txt);

2. Наблюдать ответ.

Ожидаемый результат: Сервер возвращает ошибку валидации; фронтенд показывает понятное сообщение об ошибке и не добавляет файл в библиотеку.

ЗАКЛЮЧЕНИЕ

В результате реализации проекта Meloidia создано полнофункциональное веб-приложение для комфортного прослушивания и управления аудиоколлекцией, соответствующее поставленным целям и техническим требованиям. Решение объединяет стабильный веб-плеер на React и серверную часть на Spring REST с хранением метаданных в PostgreSQL, что обеспечило целостность данных и согласованную работу всех модулей. Механизмы потокового воспроизведения и локального кэширования для оффлайн-режима доказали свою надёжность в реальных сценариях использования, а функционал создания и управления плейлистами обеспечивает удобную персонализацию музыкального опыта.

Особое значение имеет корректная интеграция фронтенда и бэкенда: REST-интерфейсы устойчиво обрабатывают загрузку и доставку аудиофайлов, а обработка ошибок сети и файловой системы минимизирует риск потери данных и обеспечивает плавный пользовательский опыт. Наличие высокопокрывающих unit-тестов (>90%) повышает уверенность в стабильности модульной логики, а интеграционные и E2E-проверки подтвердили корректность ключевых пользовательских сценариев. Текущая архитектура готова к масштабированию и дальнейшему расширению функционала при сохранении фокуса на простоте и надёжности.

Meloidia представляет собой завершённый продукт, готовый к промышленной эксплуатации в рамках заявленных ограничений. Отсутствие аутентификации и коммерческих функций было осознанно выбрано для упрощения пользовательского опыта; при необходимости эти возможности могут быть добавлены в следующих релизах. В целом проект решает задачу удобного и стабильного воспроизведения музыки с возможностью оффлайн-доступа и служит прочной основой для дальнейшего развития и улучшения пользовательских сценариев.